

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ModernHopfield(nn.Module):
    def __init__(self, dim=512, factor=1.0):
        super().__init__()
        self.dim = dim
        self.scale = dim ** -0.5 * factor
        self.beta =
nn.Parameter(torch.tensor(1.0))

    def forward(self, x): # x: [B, seq, dim]
        # Stored patterns (keys)
        keys = x
        # Query = input
        queries = x

        # Continuous Hopfield energy
        energy = torch.einsum('bnd,bmd->bnm',
queries, keys) * self.scale
        attention = F.softmax(self.beta * energy,
```

dim=-1)

```
# Retrieval: weighted sum of patterns  
retrieved = torch.einsum('bnm,bmd-  
>bnd', attention, keys)
```

```
return retrieved
```

```
# Тест: частичное воспоминание → полное  
patterns = torch.randn(10, 512) # 10  
воспоминаний  
partial = patterns[0:1] + 0.5 * torch.randn(1,  
512) # 50% шум
```

```
hopfield = ModernHopfield(512)  
reconstructed = hopfield(partial)  
cos_sim = F.cosine_similarity(reconstructed,  
patterns[0:1], dim=-1)  
print(f"Восстановление:  
{cos_sim.item():.3f}")
```